МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ

з лабораторної роботи №7
з дисципліни “Професійний Python”
на тему “Робота з базами даних у Python”

Виконала
студентка 3 курсу
групи ФеП-33с
Корхова Марія Сергіївна

Перевірив
професор
Свелеба Сергій Андрійович

Львів 2026

Мета роботи: Ознайомитися з основними принципами організації шару зберігання даних у Python застосунках, навчитися працювати з реляційними базами даних, мовою SQL, СУБД SQLite, стандартом Python DB-API, параметризованими запитами, CRUD-операціями, транзакціями, принципами ACID, патерном Repository, SQLAlchemy ORM та міграціями бази даних.

Варіант 7. Система оцінювання студентів • Продовжити проєкт Lab_01. Зберігати: ПІБ студента, дисципліну та оцінки. • Створити таблиці students, subjects, grades зі зв'язками через зовнішні ключі; реалізувати repositories для всіх сутностей. • Зберегти функції Lab_01: середній, мінімальний і максимальний бал, визначення успішності, сортування студентів, пошук студентів із середнім балом вище порога. Додати CRUD оцінок, студентів і дисциплін. • Через DB-API реалізувати параметризований вибір оцінок студента/дисципліни. Використати SQL AVG(), MIN(), MAX(). Транзакція: пакетне виставлення оцінок із rollback при некоректній оцінці. • Міграція: додати semester або assessment_type. Integration tests: CRUD, constraints оцінки, aggregates, threshold search та rollback.

Опис предметної області

Предметна область — система оцінювання студентів.

- Сутність **Студент (Student)** має унікальний ідентифікатор та ПІБ.
- Сутність **Дисципліна (Subject)** має унікальну назву.
- Сутність **Оцінка (Grade)** пов'язує студента і дисципліну, містить числове значення оцінки (value) та семестр. Зв'язок між сутностями: `Student` має багато `Grades` (one-to-many), `Subject` має багато `Grades` (one-to-many). У ORM для зв'язків задано каскад `all, delete-orphan`: при видаленні студента або дисципліни видаляються і всі пов'язані оцінки.

4. Структура проєкту

Проєкт продовжує структуру src-layout. Додано `database.py`, `db_api.py`, `repositories.py`, каталог `migrations/` та тести.

- **`database.py`**: Engine і фабрика сесій SessionLocal.
- **`models.py`**: ORM-моделі Student, Subject, Grade зі зв'язками one-to-many.
- **`repositories.py`**: Класи для виконання CRUD-операцій з моделями.
- **`services.py`**: Бізнес-логіка, пакетне виставлення оцінок (транзакція) та пошук за пороговим балом.
- **`db_api.py`**: Виконання сирих SQL-запитів без ORM (отримання AVG, MIN, MAX).
- **`migrations/`**: Конфігурація Alembic і міграція додавання поля semester.
- **`tests/`**: Fixture тестової БД (`conftest.py`) та integration tests (`test_persistence.py`).

5. Схема бази даних

(Тут ви можете намалювати просту схему від руки або текстом, як у прикладі)

- **students**: id (PK), full_name
- **subjects**: id (PK), name
- **grades**: id (PK), student_id (FK), subject_id (FK), value, semester

5.1. Таблиця students

- `id` — INTEGER (PRIMARY KEY)
- `full_name` — VARCHAR(150) (NOT NULL)

5.2. Таблиця subjects

- `id` — INTEGER (PRIMARY KEY)
- `name` — VARCHAR(100) (NOT NULL, UNIQUE)

5.3. Таблиця grades

- `id` — INTEGER (PRIMARY KEY)
- `student_id` — INTEGER (FOREIGN KEY -> students.id)
- `subject_id` — INTEGER (FOREIGN KEY -> subjects.id)
- `value` — FLOAT (NOT NULL)
- `semester` — INTEGER (NULL, додано міграцією)

6. SQL-команди створення таблиць

(Вставте SQL код створення ваших трьох таблиць, згенерований SQLAlchemy. Можете скопіювати з логів терміналу, якщо включите `echo=True` у database.py)

7. Реалізація

*У цьому розділі вставляєте код з файлів, які я вам написав у попередньому повідомленні.***7.1. database.py** *(вставити код)*

7.2. ORM-моделі (models.py) *(вставити код)*

7.3. Repository Pattern (repositories.py) *(вставити код)*

7.4. Параметризований запит через DB-API (db_api.py) *(вставити код)*

Пояснення: Функція використовує SQL-функції `AVG()`, `MIN()`, `MAX()` та працює безпосередньо з модулем `sqlite3` без ORM. Значення передаються кортежем, тому запит захищений від SQL injection. **7.5. Приклади CRUD** *(описати таблицю як у прикладі: Create, Read, Update, Delete для ваших класів)* **7.6. Сервісний шар і транзакція (services.py)** *(вставити код GradeService)* **7.7. Пояснення commit і rollback** Транзакція `assign_batch_grades` створює всі оцінки пакетно. Кожна оцінка перевіряється на коректність (від 0 до 100). Якщо виникає помилка, викликається `session.rollback()`, який скасовує всі незафіксовані зміни. Якщо все добре — `session.commit()`. **7.8. Принципи ACID у проєкті**

- **Atomicity:** Пакетне виставлення оцінок або зберігає всі оцінки, або жодної.

- **Consistency:** Обмеження NOT NULL та FOREIGN KEY не дозволяють створити оцінку для неіснуючого студента.
- **Isolation:** Транзакції ізольовані на рівні сесії SQLAlchemy.
- **Durability:** Після commit() зміни зберігаються у файлі `grades.db`.

8. Міграція Alembic

(Вставте вміст файлу міграції з папки `migrations/versions/`, який був згенерований командою alembic revision)

Зазначте, що функція `upgrade()` додала колонку `semester`, а `downgrade()` дозволяє її видалити.

9. Integration tests

(Вставте код з `conftest.py` та `test_persistence.py`)

Опишіть коротко в таблиці, що перевіряють тести (аналогічно до сторінки 20 прикладу):

- `test_crud_student` — CRUD для студента.
- `test_batch_grades_rollback` — Rollback при помилці виставлення пакетних оцінок.
- `test_threshold_search` — Пошук студентів із балом вище заданого порога.

10. Результати роботи

10.1. Запуск тестів: pytest -v*(Зробіть скріншот або скопіюйте вивід терміналу після успішного проходження pytest)*

10.2. Робота програми / Стан БД*(Зробіть кілька скріншотів бази даних (наприклад через DBeaver чи SQLite Viewer у VSCode) або логів у терміналі)*

11. Відповіді на контрольні запитання

(Просто скопіюйте всі 24 запитання і відповіді з прикладу (сторінки 22-24), вони теоретичні та універсальні для всіх варіантів)

1. Що таке база даних? Структурована сукупність даних, організована так, щоб їх можна було ефективно зберігати, отримувати, оновлювати, видаляти, фільтрувати, сортувати й агрегувати. 2. Що таке реляційна база даних? БД, у якій дані подані у вигляді взаємопов'язаних таблиць. 3. Що таке таблиця, рядок і поле? Таблиця – набір записів однакової структури; рядок (запис) – окремий об'єкт; поле (стовпець) – характеристика об'єкта. 4. Що таке SQL? Structured Query Language – мова для роботи з реляційними БД (DDL, DML, DQL, TCL). 5. Для чого використовується CREATE TABLE? Для створення таблиці із зазначенням стовпців, типів і обмежень. 6. Що означає CRUD і які SQL-команди йому відповідають? Create – INSERT, Read – SELECT, Update – UPDATE, Delete – DELETE. 7. Що таке SQLite і які її переваги? Компактна реляційна СУБД, що зберігає БД в одному файлі. Переваги: не потребує окремого сервера, проста у налаштуванні, входить до стандартної бібліотеки Python (sqlite3), зручна для тестів. 8. Що таке Python DB-API, Connection і Cursor? DB-API – стандартний інтерфейс доступу Python до реляційних БД. Connection – з'єднання з БД, що керує транзакціями; Cursor – об'єкт для виконання запитів і отримання результатів. 9. Для чого потрібні commit() і rollback()? commit() фіксує зміни транзакції, rollback() скасовує всі незафіксовані зміни. 10. Що таке SQL injection і parameterized query? Чому не слід використовувати f-string? SQL injection – впровадження шкідливого SQL через вхідні дані. Параметризований запит передає дані окремо від тексту SQL (заповнювач ?), тому вони не інтерпретуються як код. Формування SQL через f string дозволяє зловмиснику змінити запит. 11. Що таке Primary Key і Foreign Key? Primary Key однозначно ідентифікує запис таблиці. Foreign Key – поле, що посилається на ключ іншої таблиці та пов'язує таблиці (books.author_id → authors.id). 12. Що таке транзакція? Група операцій, що виконуються як єдине ціле: або всі, або жодна. 13. Що означають Atomicity, Consistency, Isolation,

Durability? Що таке ACID? Atomicity – усе або нічого; Consistency – БД залишається коректною; Isolation – транзакції не впливають некоректно одна на одну; Durability – підтверджені зміни зберігаються. ACID – абревіатура цих чотирьох властивостей. 14. Що таке persistence layer? Шар застосунку, відповідальний за збереження та читання даних. 15. Для чого потрібен Repository Pattern і чому його відокремлюють від Service Layer? Він приховує спосіб зберігання даних за єдиним інтерфейсом. 23 Сервісний шар містить бізнес-логіку й не залежить від SQLAlchemy/SQLite, тому код легше тестувати і змінювати. 16. Що таке ORM, SQLAlchemy та ORM model? ORM – відображення таблиць на Python-класи. SQLAlchemy – ORM-бібліотека для Python. ORM model – клас (Author, Book), що відповідає таблиці. 17. Що таке relationship() і one-to-many / many-to-many? relationship() описує зв'язок між моделями. One-to-many – один запис пов'язаний із багатьма (автор – книги); many-to-many – багато до багатьох, реалізується через проміжну таблицю. 18. Що таке migration і для чого Alembic? Що робить alembic upgrade head? Migration – версіонована зміна схеми БД. Alembic – інструмент міграцій для SQLAlchemy. alembic upgrade head застосовує всі міграції до останньої версії. 19. Що таке integration test і чим він відрізняється від unit test? Integration test перевіряє взаємодію кількох компонентів (репозиторій, ORM, БД), unit test – окремий модуль у ізоляції. 20. Навіщо окрема test database? Щоб тести не змінювали робочі дані, були повторюваними й незалежними. 21. Що перевіряє тест rollback? Що після помилки в транзакції у БД не залишається частково збережених даних. 22. Для чого використовуються fixtures? Для підготовки та очищення спільного оточення тестів (наприклад, створення сесії тестової БД). 23. Що робить SQL JOIN? Об'єднує рядки кількох таблиць за умовою зв'язку (books JOIN authors ON books.author_id = authors.id). 24. Що робить GROUP BY і для чого використовуються COUNT, SUM, AVG, MIN, MAX? GROUP BY

групує рядки за значенням стовпця. Агрегатні функції обчислюють кількість, суму, середнє, мінімум і максимум; у роботі використано AVG(page_count).

Висновок

У ході виконання лабораторної роботи №7 проєкт системи оцінювання студентів переведено на повноцінний persistence layer. Спроєктовано три пов'язані таблиці (`students`, `subjects`, `grades`). Реалізовано Repository Pattern, який відокремлює роботу з БД від бізнес-логіки. Продемонстровано роботу зі стандартним DB-API для SQL-агрегацій (AVG, MIN, MAX) із захистом від SQL injection. Реалізовано транзакційну операцію пакетного виставлення оцінок із `commit` та `rollback`, що забезпечує ACID. Для еволюції схеми створено міграцію Alembic, що додає поле `semester`. Усі сценарії покрито integration tests на окремій тестовій базі даних.